Projet 1 — Tarification en assurance automobile

Machine Learning 2 — Master 1 Actuariat / Big Data & Statistiques

Wilson Toussile

2026-05-31

Table des matières

Contexte 1
Données 2
Travail attendu 3
Livrables 5
Critères d'évaluation 5
Références bibliographiques 5

Contexte

La **tarification** est le cœur du métier actuariel : il s'agit d'estimer la prime juste que doit payer un assuré en fonction de son exposition au risque. En assurance automobile de responsabilité civile (RC), le risque dépend du profil du conducteur, du véhicule et du contexte géographique. Un tarif équitable doit capturer ces hétérogénéités sans surajuster (surapprentissage) ni sous-ajuster (tarif trop grossier).

La pratique industrielle repose classiquement sur des **modèles linéaires généralisés** (GLM). Ce projet explore dans quelle mesure les méthodes de machine learning du cours — régression pénalisée, méthodes locales, méthodes d'ensemble — améliorent la qualité prédictive tout en restant interprétables.

Le problème est décomposé en deux sous-modèles :

- **Modèle de fréquence** : prédire le nombre de sinistres N par unité d'exposition (nombre de sinistres par année de contrat).
- **Modèle de sévérité** : prédire le coût moyen S d'un sinistre, conditionnellement à l'occurrence d'au moins un sinistre.

La **prime pure** estimée est $\hat{\pi} = \hat{\lambda} \times \hat{s}$, où $\hat{\lambda}$ est la fréquence prédite et $\hat{s}$ la sévérité prédite.

Données

Source

Le jeu de données `freMTPL2` est un jeu de données public de référence en actuariat, issu du marché français de l'assurance RC automobile. Il est disponible :

— sur Kaggle : rechercher *"French Motor Claims freMTPL2"*
— via le package R `CASdatasets` (fonctions `freMTPL2freq` et `freMTPL2sev`)
— en exportant les deux tables en CSV pour une utilisation en Python

Table de fréquence (`freMTPL2freq`)

$n \approx 678\,000$ polices d'assurance, une ligne par contrat.

Variable	Type	Description
`IDpol`	Identifiant	Numéro de police
`ClaimNb`	Entier	Nombre de sinistres déclarés
`Exposure`	Réel $\in (0, 1]$	Durée du contrat en années
`Area`	Catégoriel (A–F)	Zone géographique (densité croissante)
`VehPower`	Entier	Puissance du véhicule (4–15)
`VehAge`	Entier	Âge du véhicule (années)
`DrivAge`	Entier	Âge du conducteur principal
`BonusMalus`	Entier	Coefficient bonus-malus (50–350)
`VehBrand`	Catégoriel (B1–B14)	Marque du véhicule
`VehGas`	Catégoriel	Carburant (Regular / Diesel)
`Density`	Entier	Densité de population de la commune
`Region`	Catégoriel (22 modalités)	Région administrative

Variable cible fréquence : taux de sinistralité $y =$ `ClaimNb`/`Exposure`.

Table de sévérité (`freMTPL2sev`)

$n \approx 26\,600$ sinistres (un par ligne, plusieurs lignes possibles par police).

Variable	Type	Description
`IDpol`	Identifiant	Clé de jointure avec `freMTPL2freq`
`ClaimAmount`	Réel	Montant du sinistre (€)

Variable cible sévérité : `ClaimAmount` (après jointure et agrégation par police).

Chargement des données

```
import pandas as pd

freq = pd.read_csv("freMTPL2freq.csv")
sev  = pd.read_csv("freMTPL2sev.csv")

# Sévérité moyenne par police (pour les polices avec au moins un sinistre)
sev_agg = sev.groupby('IDpol')['ClaimAmount'].mean().reset_index()
sev_agg.columns = ['IDpol', 'AvgSeverity']

# Jointure pour le modèle de sévérité
freq_sev = freq.merge(sev_agg, on='IDpol', how='inner')
```

Travail attendu

Partie 1 — Exploration et préparation des données

1.1 Charger et décrire les deux tables. Calculer les statistiques descriptives des variables numériques. Afficher la distribution de `ClaimNb`, `Exposure`, `BonusMalus` et `ClaimAmount`. Commenter les asymétries observées.

1.2 Calculer le taux de sinistralité moyen $\bar{\lambda} = \sum_i \texttt{ClaimNb}_i / \sum_i \texttt{Exposure}_i$. Quelle proportion de polices n'a déclaré aucun sinistre ? Quelle proportion a déclaré 2 sinistres ou plus ?

1.3 Analyser le taux de sinistralité brut par modalité pour chaque variable catégorielle (`Area`, `VehBrand`, `VehGas`, `Region`). Représenter graphiquement. Quelles variables présentent le plus fort effet différenciant ?

1.4 Construire la matrice de features X en appliquant les transformations suivantes : - Variables numériques : `VehPower`, `VehAge`, `DrivAge`, `BonusMalus`, `log(Density)` - Variables catégorielles : encodage *one-hot* (`Area`, `VehBrand`, `VehGas`) ; pour `Region`, utiliser un encodage *target encoding* (taux de sinistralité moyen par région calculé sur le train) - Tronquer `DrivAge` à 18–90 ans et `BonusMalus` à 50–300 pour éliminer les valeurs aberrantes

1.5 Découper les données en train (80 %) / test (20 %) de manière stratifiée sur `ClaimNb` (0 vs 1). Justifier l'importance du découpage stratifié ici.

Partie 2 — Modèle de fréquence

2.1 Baseline : prédire pour toute police la fréquence moyenne du train $\hat{\lambda} = \bar{\lambda}$. Calculer le RMSE et la déviance de Poisson sur le test :

$$D_{\text{Poisson}}(y, \hat{y}) = 2 \sum_i \left[y_i \ln \frac{y_i}{\hat{y}_i} - (y_i - \hat{y}_i) \right].$$

2.2 Régression Ridge et Lasso : ajuster une régression Ridge puis Lasso (avec `LassoCV`) sur le taux de sinistralité $y =$ `ClaimNb/Exposure`, en pondérant chaque observation par `Exposure`. Tracer le chemin de régularisation Lasso. Quelles variables sont éliminées en premier ? Calculer RMSE test pour les deux méthodes.

2.3 Arbre de décision CART : ajuster un arbre sur la fréquence. Explorer l'effet de la profondeur maximale (`max_depth` $\{3, 5, 8, 15\}$) sur les erreurs train et test. Afficher l'arbre pour `max_depth=4` et interpréter les premières coupures.

2.4 Forêt aléatoire : ajuster une `RandomForestRegressor`. Tracer la courbe d'erreur OOB en fonction du nombre d'arbres. Afficher l'importance des variables (impureté de Gini). Comparer avec les coefficients du Lasso : les deux méthodes s'accordent-elles sur les variables les plus importantes ?

2.5 Gradient Boosting : ajuster un `GradientBoostingRegressor` ou `HistGradientBoostingRegressor`. Sélectionner les hyperparamètres (`n_estimators`, `learning_rate`, `max_depth`) par validation croisée 5-fold. Tracer les courbes de perte train/validation.

2.6 Tableau comparatif : reporter RMSE test et déviance de Poisson test pour toutes les méthodes. Conclure sur la méthode la plus performante et la plus interprétable.

Partie 3 — Modèle de sévérité

3.1 Restreindre aux polices ayant au moins un sinistre (`ClaimNb` `1`). Afficher la distribution de `AvgSeverity` et de `log(AvgSeverity)`. Justifier le recours à la transformation logarithmique.

3.2 Ajuster les mêmes méthodes qu'en Partie 2 (Lasso, forêt aléatoire, gradient boosting) sur `log(AvgSeverity)`. Calculer le RMSE en échelle log et en échelle originale (après ré-exponentiation et correction du biais : $\hat{s} = \exp(\hat{y} + \hat{\sigma}^2/2)$).

3.3 Analyser l'importance des variables pour la sévérité. Les mêmes variables sont-elles importantes pour la fréquence et la sévérité ? Interpréter la différence éventuelle.

Partie 4 — Prime pure et analyse

4.1 Calculer la prime pure $\hat{\pi}_i = \hat{\lambda}_i \times \hat{s}_i$ pour toutes les polices du test (en utilisant la forêt aléatoire ou le gradient boosting comme meilleur modèle de chaque partie). Afficher la distribution des primes pures prédites.

4.2 Segmenter les assurés en déciles de prime pure prédite. Pour chaque décile, calculer la prime pure réelle observée (`ClaimNb × AvgSeverity / Exposure`). Tracer le **graphe de Lorenz actuariel** (prime prédite vs prime observée par décile). Une courbe proche de la diagonale indique un bon classement des risques.

4.3 Identifier les 1 % de polices les plus risquées selon le modèle. Quel est leur taux de sinistralité réel dans le test ? Comparer à la moyenne globale. Ce ratio correspond au **rapport de concentration** du modèle.

4.4 Discussion critique : - Quelles sont les limites de l'approche fréquence × sévérité ? - Comment la réglementation (non-discrimination par sexe, RGPD) contraint-elle le choix des variables ? - En quoi l'interprétabilité (Lasso, arbre) est-elle un enjeu en assurance par rapport à la pure performance prédictive ?

Livrables

Livrable	Format	Attendu
Notebook Python	`.ipynb` ou `.qmd`	Reproductible, cellules documentées, figures lisibles
Rapport	PDF, 5–8 pages	Problématique, méthodes, résultats, discussion
Présentation	Diaporama, 10 min	Problème → données → résultats → conclusion

Le notebook doit pouvoir s'exécuter de bout en bout sur la machine de l'évaluateur après `pip install -r requirements.txt`.

Critères d'évaluation

Critère	Points
Qualité de l'exploration et de la préparation des données	4
Justesse de l'implémentation des méthodes	5
Pertinence de l'évaluation comparative (métriques, protocole)	4
Qualité de l'interprétation métier	4
Clarté du rapport et de la présentation orale	3
Total	**20**

Références bibliographiques

— Hastie, Tibshirani et Friedman [3] — chapitres 3 (régression pénalisée) et 15 (forêts aléatoires)
— Friedman [2] — gradient boosting
— Breiman [1] — forêts aléatoires
— Noll, A., Salzmann, R., Wüthrich, M.V. (2020). *Case Study : French Motor Third-Party Liability Claims*. SSRN 3164764.

Références

[1] Leo Breiman. "Random Forests". In : *Machine Learning* 45.1 (2001), p. 5-32.

[2] Jerome H. Friedman. "Greedy Function Approximation : A Gradient Boosting Machine". In : *The Annals of Statistics* 29.5 (2001), p. 1189-1232.

[3] Trevor HASTIE, Robert TIBSHIRANI et Jerome FRIEDMAN. *The Elements of Statistical Learning : Data Mining, Inference, and Prediction*. 2e éd. New York : Springer, 2009. URL : https://web.stanford.edu/~hastie/ElemStatLearn/.